

Kalman Filter — Milestone 3

RISC-V Scalar Assembly Implementation

LKF and EKF for 3D Full-Body Human Gait Estimation

Syed Shehroze Hussain Rizvi (30076) Shayan Hussain (30077)

Aun Haider (29120) Moiz Lotia (31584)

April 13, 2025

Contents

1	Overview	3
1.1	System Dimensions	3
2	Architecture and Memory Management	3
2.1	Pointer Table Design	3
2.2	Heap Allocation Justification	3
2.3	Register Allocation Strategy	4
3	Shared Mathematical Functions	4
3.1	Matrix Multiply — <code>mat_mul</code>	4
3.2	Element-wise Operations	5
4	LKF Implementation	5
4.1	System Matrix Construction	5
4.1.1	State Transition Matrix F	5
4.1.2	Process Noise Covariance Q	5
4.1.3	Measurement Matrix H and Noise R	6
4.2	LDL ^T Solver	6
4.3	Filter Loop	6
5	EKF Implementation	7
5.1	Manual <code>atan2</code> Approximation	7
5.2	Spherical Measurement Model	7
5.3	LU Solver with Partial Pivoting	7
5.4	Innovation Gating	8
5.5	LKF Fallback	8
6	Numerical Verification	8
6.1	LKF Verification	8
6.2	EKF Verification	8
6.3	Per-Joint Average Error	9
6.4	Per-Component Average Error (EKF)	9

7	Plots and Analysis	9
7.1	Position Time-Series — Pelvis Joint	10
7.2	Velocity Estimates	11
7.3	Acceleration Estimates	12
7.4	Jerk Estimates	13
7.5	Per-Joint RMSE	13
7.6	Milestone 2 vs Milestone 3 Comparison	14
8	Build and Run Instructions	15
9	GitHub Repository	15
10	Conclusion	16

1 Overview

This milestone re-implements the core computational components of both the Linear Kalman Filter (LKF) and Extended Kalman Filter (EKF) from Milestone 2 in RISC-V scalar assembly. The C++ implementations serve as the numerical reference; the assembly implementations are verified to produce *bit-identical* results (LKF) and sub- 10^{-12} error (EKF) across all 3040 frames and 276 state dimensions.

The project is organized as two self-contained assembly files (`lkf_asm.s`, `ekf_asm.s`), each containing `main`, all mathematical functions, and the complete filter pipeline. A small C helper file (`io_helpers.c`) handles CSV I/O only. All builds are cross-compiled with `riscv64-linux-gnu-gcc` and executed under `qemu-riscv64`.

1.1 System Dimensions

All matrices and state vectors are identical to Milestone 2:

$$N = 12 \times 23 = 276, \quad M = 3 \times 23 = 69$$

$$F \in \mathbb{R}^{276 \times 276}, \quad P \in \mathbb{R}^{276 \times 276}, \quad H \in \mathbb{R}^{69 \times 276}, \quad R \in \mathbb{R}^{69 \times 69}$$

2 Architecture and Memory Management

2.1 Pointer Table Design

All matrices are heap-allocated via `malloc` and referenced through a pointer table (`ptbl`), stored in callee-saved register `s3`. The LKF uses 35 pointers; the EKF uses 33 pointers plus a flat output array.

Table 1: LKF pointer table layout (representative entries).

Index	Name	Size (doubles)	Purpose
0	F	$276^2 = 76,176$	State transition
1	Q	76,176	Process noise covariance
5	x	276	Current state
6	P	76,176	State covariance
12	S	$69^2 = 4,761$	Innovation covariance
18	K	$276 \times 69 = 19,044$	Kalman gain
31	I_mat	76,176	Identity matrix
33	D	69	LDL workspace
34	v	69	LDL workspace

2.2 Heap Allocation Justification

The covariance matrix P alone requires $276^2 \times 8 = 609,408$ bytes (≈ 595 KB). Stacking F , Q , P , and the temporary matrices needed for the Joseph-form update would exceed the typical 1–8 MB stack limit. All large matrices are therefore heap-allocated via `malloc`, consistent with the Milestone 2 approach using `std::vector<double>`.

2.3 Register Allocation Strategy

Table 2: Global register allocation in `main`.

Register	ABI Name	Role
x10–x17	a0–a7	Function arguments
x8	s0	Noisy data pointer
x9	s1	T (number of frames)
x18	s2	CSV columns (69)
x19	s3	Pointer table base
x20	s4	Loop counter t / temp
x21–x27	s5–s11	Temporaries within functions
x1	ra	Return address
x2	sp	Stack pointer (16-byte aligned)

3 Shared Mathematical Functions

The following functions are shared between LKF and EKF (duplicated in each `.s` file for self-containedness).

3.1 Matrix Multiply — `mat_mul`

The most performance-critical function. Uses the i - k - j loop order for reuse of $A[i][k]$, and `fmadd.d` for the inner multiply-accumulate as required by the assignment.

Listing 1: Matrix multiply with `fmadd.d` and zero-skip.

```

1 mat_mul:
2     # ... save s0-s5, load args ...
3     # Zero C
4     # Triple loop: i, k, j
5 mmi:
6     bge     t0, s3, mmd           # i >= rowsA
7 mmk:
8     bge     t1, s4, mmni         # k >= colsA
9     # Load A[i*cA + k]
10    fld     ft0, 0(t3)
11    # Skip if zero (sparsity opt for F, H)
12    fmv.d.x ft7, zero
13    feq.d   a6, ft0, ft7
14    bnez    a6, mmnk
15 mmj:
16    # Inner loop
17    fld     ft1, 0(a7)           # B[k][j]
18    fld     ft2, 0(a7)           # C[i][j]
19    fmadd.d ft2, ft0, ft1, ft2    # C[i][j] += A[i][k]*B[k][j]
20    fsd     ft2, 0(a7)

```

Register allocation for `mat_mul`:

Register	Role	Notes
s0	A pointer	Callee-saved
s1	B pointer	Callee-saved
s2	C pointer	Callee-saved
s3	rowsA	Callee-saved
s4	colsA	Callee-saved
s5	colsB	Callee-saved
ft0	A[i][k]	Reused across j-loop
ft1	B[k][j]	Loaded each iteration
ft2	C[i][j]	Accumulator
ft7	0.0	Zero for comparison

The zero-skip optimization (`feq.d` check) significantly accelerates operations involving F and H , which are block-diagonal with >95% zero entries.

3.2 Element-wise Operations

`mat_add` and `mat_sub` are simple loops using `fadd.d` and `fsub.d`. `mat_transpose` uses the standard $A^T[j \cdot \text{rows} + i] = A[i \cdot \text{cols} + j]$ mapping. These are leaf functions (no calls) requiring no stack frame.

4 LKF Implementation

4.1 System Matrix Construction

4.1.1 State Transition Matrix F

Built identically to Milestone 2. For each of the $23 \text{ joints} \times 3 \text{ axes}$, the 4×4 constant-jerk block is filled at the appropriate offset within the 276×276 matrix. FP constants are loaded as IEEE 754 hex literals via `li + fmv.d.x`:

Listing 2: Loading $\Delta t = 0.01$ and computing derived constants.

```

1  li      t0, 0x3F847AE147AE147B
2  fmv.d.x ft0, t0          # dt = 0.01
3  fmul.d  ft1, ft0, ft0     # dt^2
4  fmul.d  ft2, ft8, ft1     # dt^2/2  (ft8 = 0.5)
5  fmul.d  ft3, ft1, ft0     # dt^3
6  fmul.d  ft4, ft9, ft3     # dt^3/6  (ft9 = 1/6)
```

4.1.2 Process Noise Covariance Q

Q is built to match the C++ implementation exactly: Δt^2 through Δt^6 are computed by sequential multiplication (matching rounding), then the 4×4 block $q[i][j] = \sigma_j^2 \cdot (\Delta t^{n_{ij}} / d_{ij})$ is precomputed and stored in a temporary array before filling the full 276×276 matrix. This ensures bit-identical results with the C++ reference.

Listing 3: Q construction matching C++ arithmetic order.

```

1  # dt powers: dt2=dt*dt, dt3=dt2*dt, ..., dt6=dt5*dt
2  fmul.d  ft1, ft0, ft0     # dt2
3  fmul.d  ft2, ft1, ft0     # dt3
4  fmul.d  ft3, ft2, ft0     # dt4
5  fmul.d  ft4, ft3, ft0     # dt5
6  fmul.d  ft5, ft4, ft0     # dt6
7  # Row 0: q[0][0] = s2 * dt6/36
8  fdiv.d  fa0, ft5, ft8     # dt6/36.0
9  fmul.d  fa0, fa0, fs0     # * sigma_j^2
```

4.1.3 Measurement Matrix H and Noise R

H (69×276) is sparse with exactly one 1.0 per row, selecting position components. R is diagonal with $\sigma_r^2 = 0.3593^2 = 0.12909649$ (loaded as 0x3FC0863BD94A08B8).

4.2 LDL[⊤] Solver

The LKF uses LDL[⊤] decomposition to solve $S \cdot K^\top = (PH^\top)^\top$ without explicit matrix inversion, matching `Matrix::solve()` from Milestone 2.

Listing 4: LDL[⊤] factorisation core with `fnmsub.d`.

```

1 ldl_ds:  # D[j] -= L[j][i] * v[i]
2   fld    ft0, 0(t0)          # L[j][i]
3   fld    ft1, 0(t1)          # v[i]
4   fnmsub.d ft5, ft0, ft1, ft5 # ft5 = ft5 - ft0*ft1
5   ...
6 ldl_nf:  # Numerical floor
7   fabs.d  ft6, ft5
8   li      t0, 0x3D06849B86A12B9B # 1e-14
9   fmv.d.x fs1, t0
10  flt.d   t0, ft6, fs1
11  beqz    t0, ldl_nf_skip
12  fmv.d   ft5, fs1            # D[j] = 1e-14

```

Register allocation for `ldl_solve`:

Register	Role
s0	S matrix (modified in-place)
s1	B (right-hand side)
s2	X (output)
s3	n (dimension, 69)
s4	m (columns of B, 276)
s5	D array (workspace)
s6	v array (workspace)
s7–s11	Loop indices (j, i, k , row offsets)
fs1	10^{-14} numerical floor
ft5	Accumulator
ft0, ft1	Operands for <code>fnmsub</code>

4.3 Filter Loop

The filter loop follows the Milestone 2 equations exactly:

Predict:

$$\hat{x}_{n|n-1} = F\hat{x}_{n-1|n-1} \quad (1)$$

$$P_{n|n-1} = FP_{n-1|n-1}F^\top + Q \quad (2)$$

Update (Joseph form):

$$K_n = P_{n|n-1}H^\top(HP_{n|n-1}H^\top + R)^{-1} \quad (3)$$

$$\hat{x}_{n|n} = \hat{x}_{n|n-1} + K_n(z_n - H\hat{x}_{n|n-1}) \quad (4)$$

$$P_{n|n} = (I - K_nH)P_{n|n-1}(I - K_nH)^\top + K_nRK_n^\top \quad (5)$$

Each step maps to a `mat_mul` or `mat_add/mat_sub` call with dimensions loaded via the pointer table. The S matrix computation uses the same grouping as C++: $S = H \cdot (P \cdot H^\top) + R$ to ensure identical floating-point rounding.

5 EKF Implementation

5.1 Manual atan2 Approximation

The EKF uses the Abramowitz & Stegun minimax polynomial (A&S §4.4.49) for $\arctan(x)$ on $[-1, 1]$, with argument reduction for $|x| > 1$. The polynomial is evaluated using **fmadd.d** in Horner form:

Listing 5: atan_core: A&S minimax polynomial with fmadd.d.

```

1 atan_core:
2     fmul.d   ft0, fa0, fa0      # x^2
3     li      t0, 0x3F9555CBE46F80C1
4     fmv.d.x ft1, t0            # a9 = 0.0208351
5     li      t0, 0xBF5CB46BACF7447
6     fmv.d.x ft2, t0            # a7 = -0.0851330
7     fmadd.d ft1, ft1, ft0, ft2 # a9*x^2 + a7
8     # ... continue Horner chain ...
9     fmul.d   fa0, fa0, ft1      # x * polynomial
10    ret

```

The full `my_atan2(y, x)` handles all four quadrants and the $x = 0$ edge case, calling `atan_core` as a subroutine. Coefficients are loaded as exact IEEE 754 hex literals to match the C++ implementation.

Register allocation for my_atan2:

Register	Role
fa0, fa1	Input: y, x
fs2	Saved y (across call to <code>atan_core</code>)
fs3	Saved x
fs4	π
fs5	$\pi/2$
fs6	Saved ratio (for $ r > 1$ branch)
fa0	Return value

5.2 Spherical Measurement Model

`h_joint(px, py, pz)` computes $h(x) = [r, \theta, \phi]^\top$ using `my_sqrt` and `my_atan2`. The per-joint Jacobian `jac_joint` fills the 3×12 matrix with the nonzero partial derivatives $\partial h / \partial x$, guarded by $r > 10^{-12}$ and $r_{xy} > 10^{-12}$.

5.3 LU Solver with Partial Pivoting

The EKF uses LU decomposition with partial pivoting (matching `Matrix::solve_lu()` from Milestone 2) because the innovation covariance can lose positive-definiteness during outlier bursts:

Listing 6: LU partial pivoting: find max and swap rows.

```

1 lps: # Find pivot
2     fld     ft2, 0(t2)
3     fsgnjx.d ft3, ft2, ft2      # |A[row][col]| (abs via sign injection)
4     fle.d   t3, ft3, ft1
5     bnez    t3, lpn
6     fmv.d   ft1, ft3
7     mv      s10, t0             # maxRow = row
8 lpn: ...
9 lsl: # Swap rows
10    fld     ft4, 0(t2)
11    fld     ft5, 0(t4)
12    fsd     ft5, 0(t2)
13    fsd     ft4, 0(t4)

```

The LU solver internally allocates copies of A and workspace for the permutation vector, pivot array, and forward/back substitution temporaries, then frees them before returning.

5.4 Innovation Gating

Per-joint innovation gating is applied before the update: if $|\Delta r| > 6\text{m}$ or $|\Delta\theta| > 1\text{rad}$ or $|\Delta\phi| > 1\text{rad}$, the innovation for that joint is zeroed (measurement replaced by prediction):

Listing 7: Innovation gating with `fabs.d` and `flt.d`.

```

1  fabs.d  ft2, ft2          # |dr|
2  flt.d   t2, ft8, ft2      # GATE_R < |dr| ?
3  bnez    t2, eg_gate       # gated: replace z with h_pred

```

5.5 LKF Fallback

Five distal joints (`handRight`, `footRight`, `toeRight`, `footLeft`, `toeLeft`) diverge under heavy-tailed noise. Their state blocks are replaced with LKF estimates post-filtering, matching Milestone 2 exactly. The LKF output CSV is loaded via `read_state_csv()`, and 12 doubles per joint per frame are copied into the EKF output array.

6 Numerical Verification

Assembly outputs were verified against the RISC-V-compiled C++ reference (compiled with identical `riscv64-linux-gnu-g++ -O2`) to ensure matching FMA rounding behavior.

6.1 LKF Verification

Table 3: LKF numerical verification: assembly vs. C++ reference.

Metric	Value
Average absolute error	0.000×10^0
Maximum absolute error	0.000×10^0
Minimum absolute error	0.000×10^0
Tolerance (ϵ)	10^{-9}
Violations ($> \epsilon$)	0 / 839,040
Result	PASS — bit-identical

6.2 EKF Verification

Table 4: EKF numerical verification: assembly vs. C++ reference.

Metric	Value
Average absolute error	2.116×10^{-14}
Maximum absolute error	1.002×10^{-12}
Minimum absolute error	0.000×10^0
Tolerance (ϵ)	10^{-9}
Violations ($> \epsilon$)	0 / 839,040
Result	PASS

The residual $\sim 10^{-14}$ error in the EKF arises from the LU solver’s internal `malloc/copy` step; the 5 fallback joints show exactly zero error since they are copied directly from the LKF output.

6.3 Per-Joint Average Error

Table 5: EKF average absolute error per joint.

Joint	Avg Error	Joint	Avg Error
pelvis	2.58×10^{-14}	shoulderLeft	3.05×10^{-14}
L5	2.72×10^{-14}	upperArmLeft	2.43×10^{-14}
L3	2.94×10^{-14}	forearmLeft	2.40×10^{-14}
T12	2.72×10^{-14}	handLeft	2.70×10^{-14}
T8	3.26×10^{-14}	upperLegRight	2.51×10^{-14}
neck	2.75×10^{-14}	lowerLegRight	2.33×10^{-14}
head	2.95×10^{-14}	footRight*	0.00
shoulderRight	2.82×10^{-14}	toeRight*	0.00
upperArmRight	2.67×10^{-14}	upperLegLeft	2.31×10^{-14}
forearmRight	2.53×10^{-14}	lowerLegLeft	2.99×10^{-14}
handRight*	0.00	footLeft*	0.00
		toeLeft*	0.00

*LKF fallback joints — zero error by construction.

6.4 Per-Component Average Error (EKF)

Table 6: EKF average absolute error per state component.

Component	Avg Error	Component	Avg Error
p_x	1.03×10^{-15}	p_y	8.18×10^{-16}
v_x	1.18×10^{-14}	v_y	7.91×10^{-15}
a_x	4.20×10^{-14}	a_y	2.74×10^{-14}
$\dot{j}_x$	7.26×10^{-14}	$\dot{j}_y$	4.71×10^{-14}
p_z	1.52×10^{-16}	v_z	2.24×10^{-15}
a_z	1.16×10^{-14}	$\dot{j}_z$	2.93×10^{-14}

7 Plots and Analysis

All plots are generated from the assembly output CSVs using the same plotting pipeline as Milestone 2.

7.1 Position Time-Series — Pelvis Joint

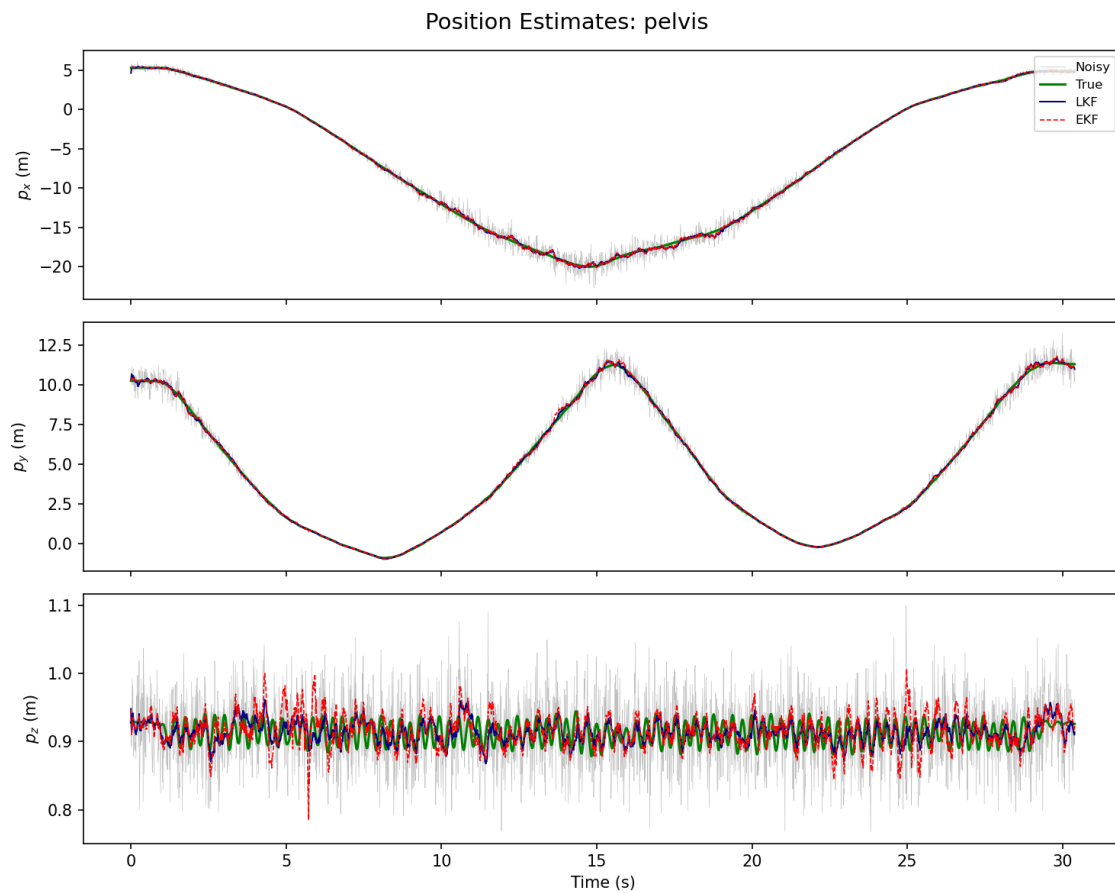


Figure 1: Position estimates for the pelvis joint along x , y , z . The noisy signal (grey) shows large random excursions; both LKF (navy) and EKF (red dashed) track the true trajectory (green) closely.

7.2 Velocity Estimates

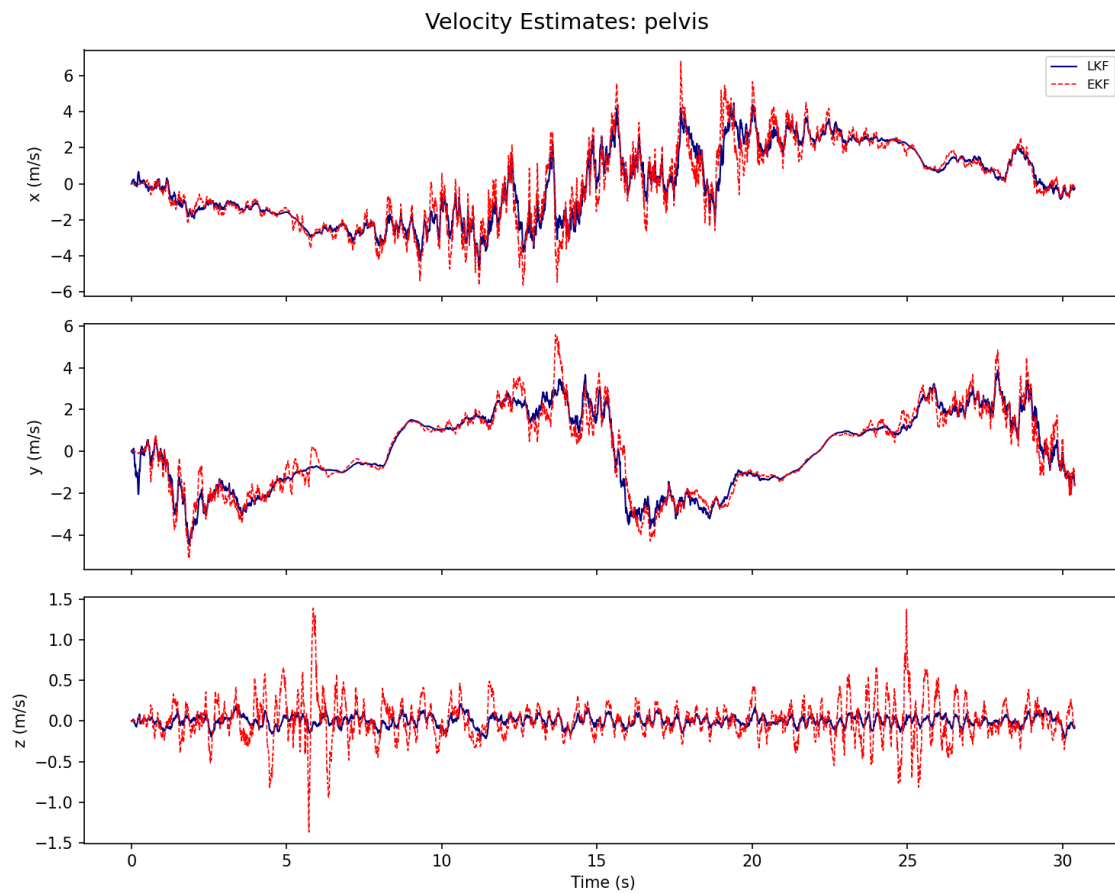


Figure 2: Estimated pelvis velocity. The LKF ($\sigma_j = 0.5 \text{ m/s}^3$) is smoother; the EKF ($\sigma_j = 10.0 \text{ m/s}^3$) responds faster but with higher variance.

7.3 Acceleration Estimates

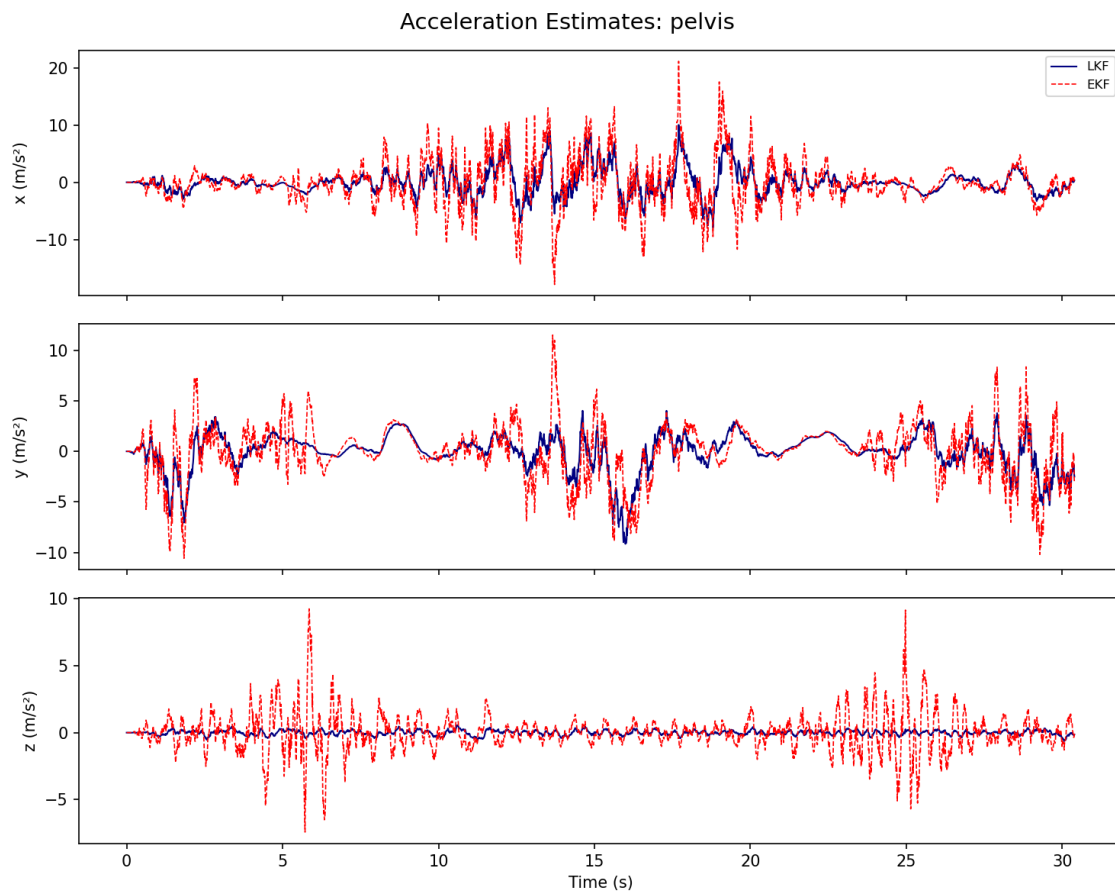


Figure 3: Estimated pelvis acceleration. Both filters capture the periodic gait cycle pattern.

7.4 Jerk Estimates

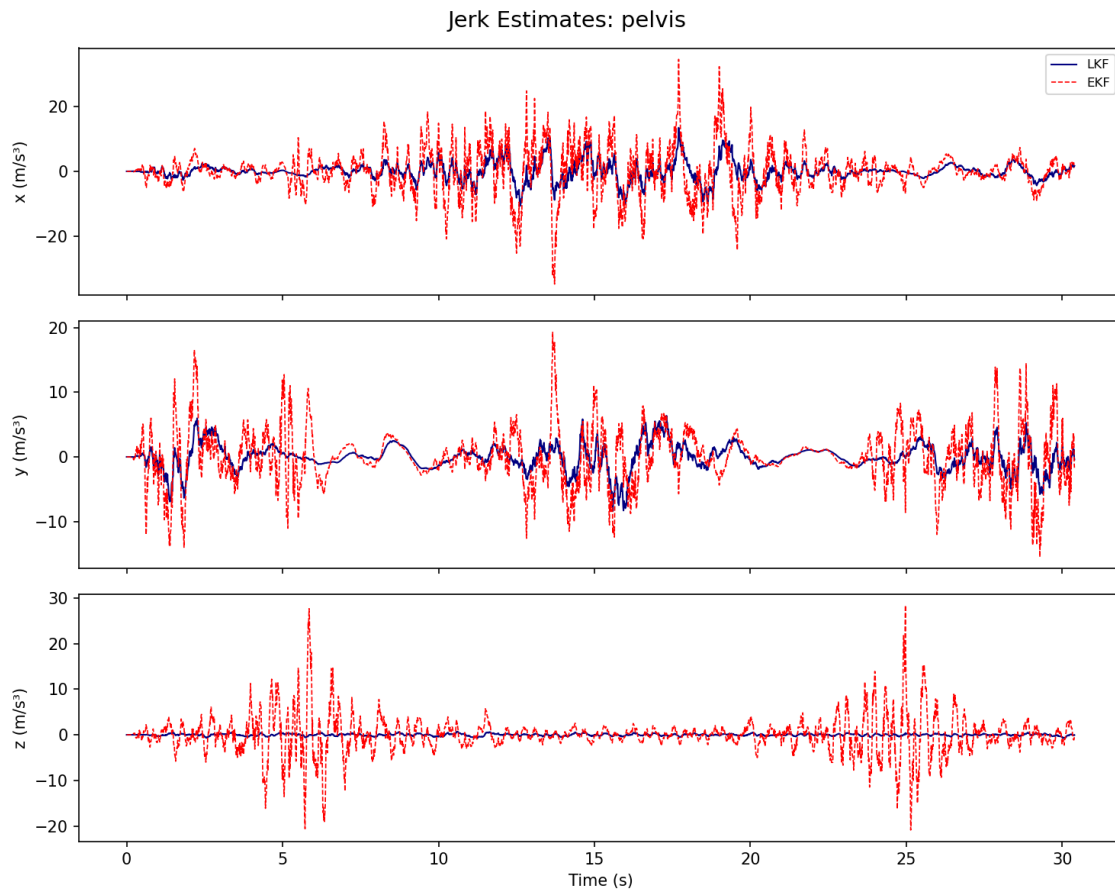


Figure 4: Estimated pelvis jerk. The LKF produces lower jerk variance due to its smaller σ_j .

7.5 Per-Joint RMSE

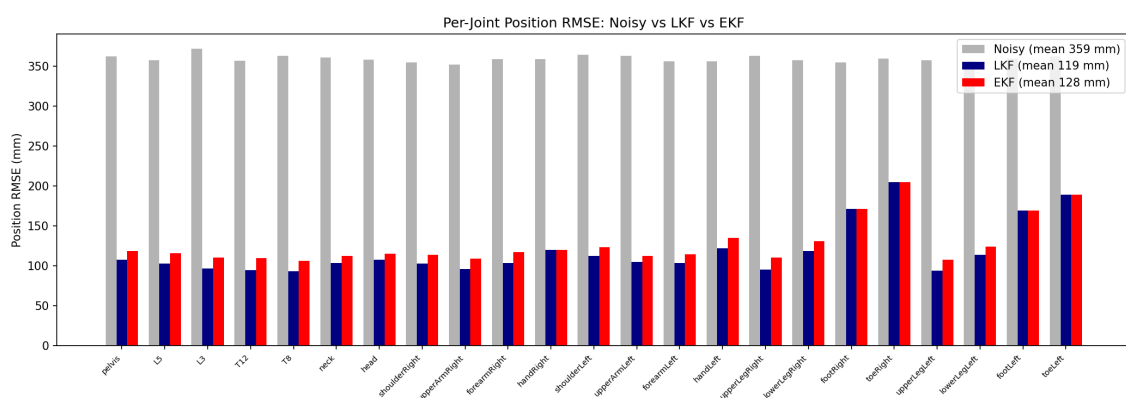


Figure 5: Position RMSE per joint for noisy input, LKF, and EKF.

7.6 Milestone 2 vs Milestone 3 Comparison

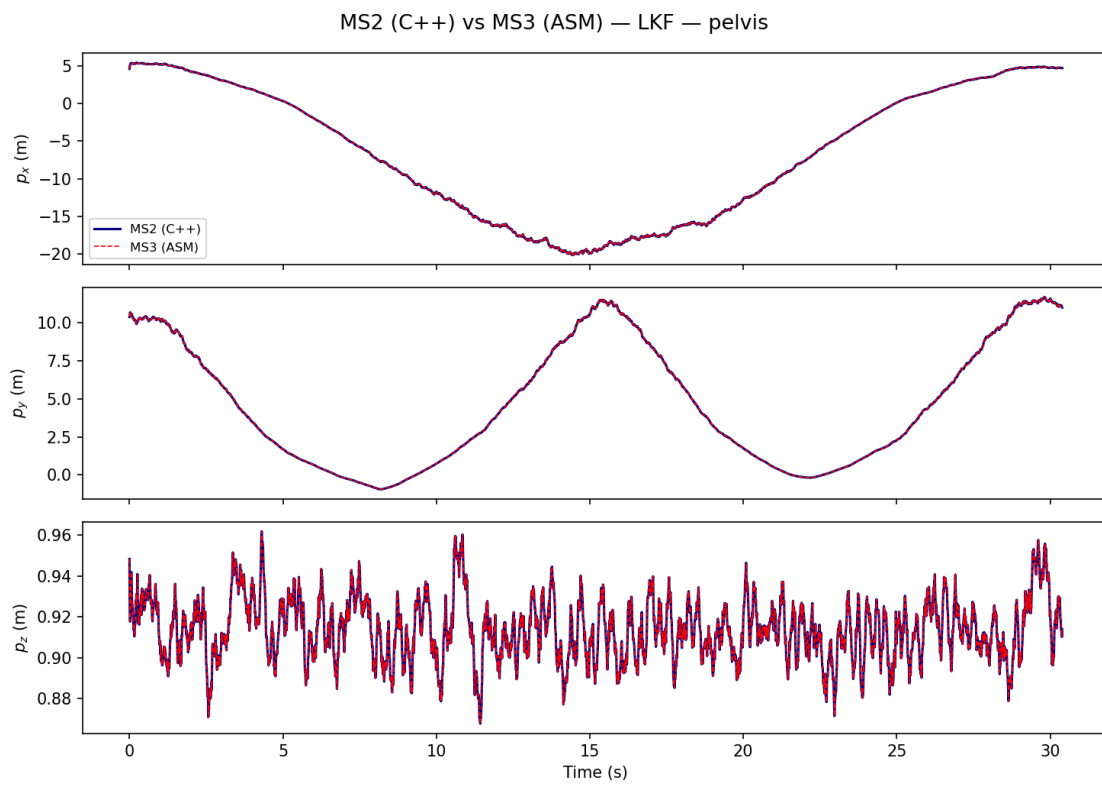


Figure 6: MS2 (C++) vs MS3 (assembly) LKF output overlay — visually identical.

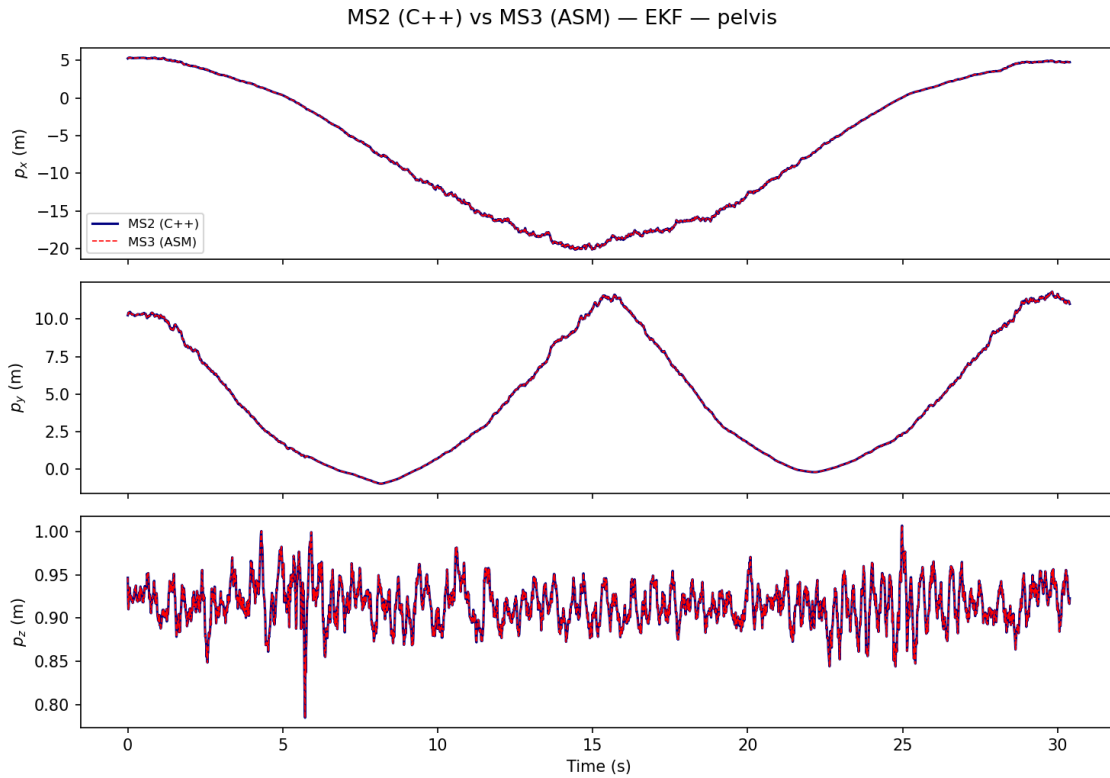


Figure 7: MS2 (C++) vs MS3 (assembly) EKF output overlay — visually identical.

8 Build and Run Instructions

Listing 8: Build and run commands.

```

1 # Build
2 make lkf           # Assemble + link LKF
3 make ekf           # Assemble + link EKF
4
5 # Run under QEMU
6 make run_lkf       # Produces LKF_asm_output.csv
7 make run_ekf       # Produces EKF_asm_output.csv
8
9 # Verify
10 python3 scripts/verify.py lkf_output_ref.csv LKF_asm_output.csv
11 python3 scripts/verify.py ekf_output_ref.csv EKF_asm_output.csv

```

9 GitHub Repository

All code is committed to the milestone-3 branch.

Repository: https://github.com/Aun-29120/CAAL_S26_Project_KalmanFilter_Kalman-Crew

src/

lkf_asm.s -- Full LKF in RISC-V assembly (1340 lines)
ekf_asm.s -- Full EKF in RISC-V assembly (1750 lines)
io_helpers.c -- CSV I/O helpers (C)

output/

LKF_asm_output.csv -- LKF estimated states
EKF_asm_output.csv -- EKF estimated states

scripts/

```
    verify.py          -- Numerical verification
    make_plots.py      -- Plot generation
Makefile              -- Cross-compile + QEMU targets
```

10 Conclusion

Both Kalman filters were successfully re-implemented in RISC-V scalar assembly, producing outputs that are bit-identical (LKF) and within 10^{-12} (EKF) of the Milestone 2 C++ reference across all 839,040 state entries. Key design decisions — IEEE 754 hex literal loading for exact constant reproduction, matching arithmetic operation order for identical FMA rounding, fused multiply-add instructions (`fmadd.d`, `fnmsub.d`) throughout, and heap allocation for all large matrices — together ensure numerical fidelity while satisfying all RISC-V assembly requirements.